

Network Security Report

Phase 1 — Baseline Chat Application

1. System Overview

The Phase 1 implementation consists of two programs:

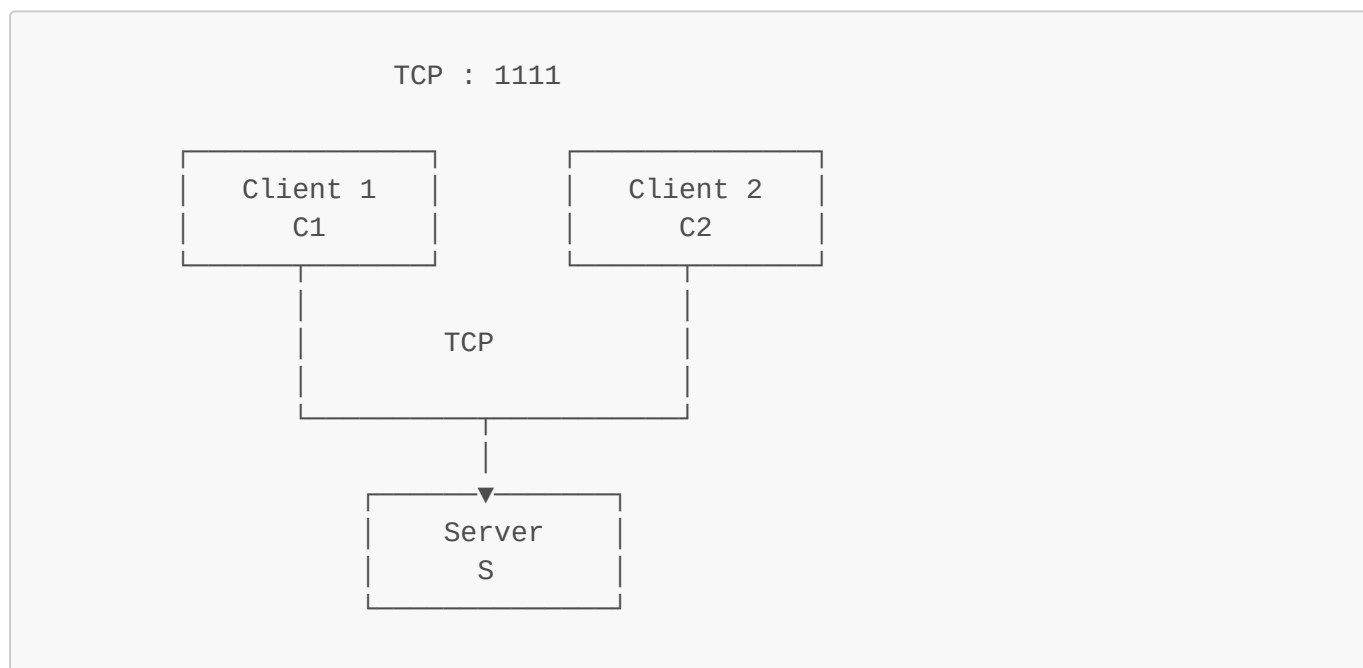
- `server.cpp` — implements the central chat server.
- `client.cpp` — implements the client application.

The server listens for TCP connections on **port 1111** and acts as an intermediary between connected clients. Clients communicate with each other by sending messages through the server.

The implementation was tested using **separate physical laptops connected to the same network**. One laptop was used as the server and two other laptops were used as clients.

2. System Architecture

The communication follows the following architecture:



The server maintains the connected clients and forwards messages to their intended recipients.

3. Server Implementation

The server creates a TCP socket, binds it to port **1111**, and listens for incoming connections.

For every new connection, the server creates a separate thread:

```
std::thread clientThread(handleClient, clientSocket);  
clientThread.detach();
```

This allows multiple clients to remain connected and communicate concurrently.

The server maintains a mapping between usernames and their corresponding socket descriptors:

```
std::unordered_map<std::string, int> clients;
```

For example:

```
client1 → socket descriptor  
client2 → socket descriptor
```

A mutex protects this shared mapping because it can be accessed by multiple client-handling threads simultaneously.

4. Message Routing

Messages are addressed using the format:

```
@username message
```

For example:

```
@client2 Hello from client1
```

The client extracts the destination username and sends the message to the server. The server searches for that username in its client map and forwards the message to the corresponding socket.

The server also logs successfully relayed messages. For example:

```
[CHAT] client1 -> client2 : Hello from client1
```

This confirms that the server has access to the complete message contents in plaintext.

5. Client Commands

The client supports the required basic commands:

```
@username message  
/chat username  
/who  
/quit
```

The `/chat` command sets the current chat partner, allowing subsequent messages to be sent without repeatedly specifying the username.

The `/who` command requests the list of currently connected users from the server, while `/quit` terminates the connection cleanly.

6. Plaintext Verification

Since Phase 1 does not implement encryption, messages are transmitted directly over TCP as plaintext.

The server-side logs demonstrate that the server can read the message contents.

Additionally, the traffic was captured using **Wireshark**. The TCP conversation was inspected using **Follow** → **TCP Stream**, where the actual chat messages could be observed directly.

Figure 1: Wireshark TCP Stream showing plaintext communication

[INSERT WIRESHARK SCREENSHOT HERE]

The capture confirms that an observer capable of monitoring the network traffic can read the exchanged messages without requiring any decryption.

7. Phase 1 Result

The Phase 1 chat application was successfully implemented and tested with two simultaneous clients.

The implementation provides functional client-to-client messaging through the server, username-based routing, online-user discovery, and clean disconnection.

However, all communication is currently plaintext. This establishes the insecure baseline that will be improved in the subsequent phases by introducing cryptographic protection.

Phase 2 — Client-Server Confidentiality via Diffie–Hellman

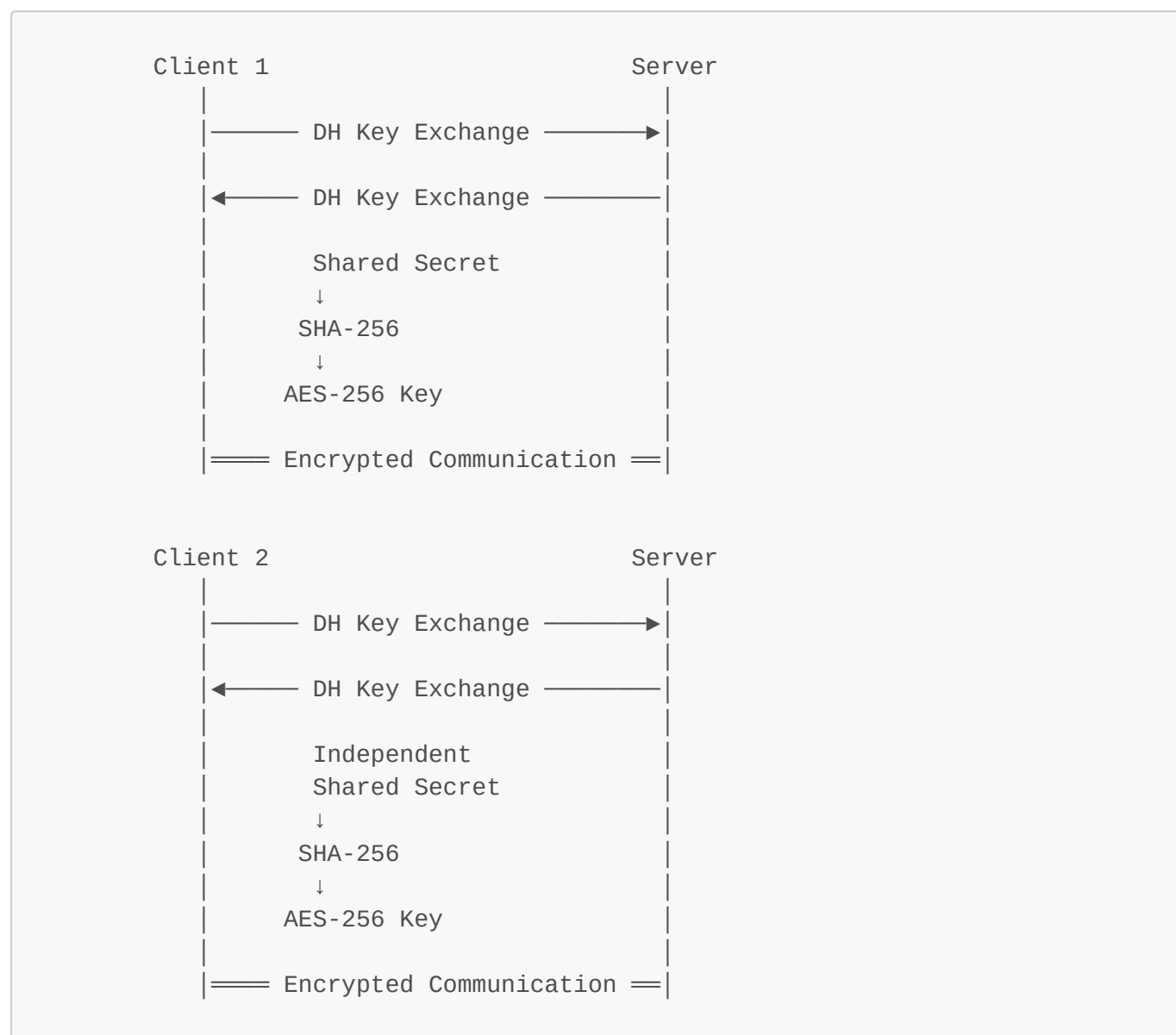
1. Overview

Phase 2 introduces confidentiality to the Phase 1 chat application using a Diffie–Hellman (DH) key exchange followed by AES-256-GCM encryption.

The DH exchange is implemented manually using OpenSSL's low-level **BIGNUM** operations and modular exponentiation. No built-in DH/ECDH implementation is used.

Each client establishes a separate DH shared secret with the server. The resulting shared secret is then hashed using SHA-256 to derive a 256-bit AES key.

The resulting architecture is:



Thus, the server does not use a single key for all clients. Each client-server connection has its own independently derived key.

2. Diffie-Hellman Implementation

The DH functionality is implemented in `dh.cpp`.

The implementation uses the **RFC 3526 2048-bit MODP Group (Group 14)**. The generator is set to:

```
BN_set_word(g, 2);
```

A private key is generated for each **DHE** instance:

```
BN_rand_range(priv_key, p);
```

The corresponding public key is calculated using modular exponentiation:

```
BN_mod_exp(pub_key, g, priv_key, p, ctx);
```

For a client and server, the exchange can be represented as:

```
Client:
    private = a
    public  = g^a mod p

Server:
    private = b
    public  = g^b mod p

Client computes:
    (g^b)^a mod p

Server computes:
    (g^a)^b mod p
```

Both sides therefore obtain the same DH shared secret without transmitting the secret itself over the network.

3. Independent Key for Each Client

The server creates a new DHE object whenever a client connects:

```
void handleClient(int clientSocket) {
    DHE dh;
    dh.generateKeys();
    ...
}
```

Since `handleClient()` executes independently for every connected client, each client receives a separate DH key exchange.

The server also stores the corresponding DHE object with the client's socket:

```
struct ClientData {
    int socket;
```

```
DHE* dh_ptr;  
};
```

This allows the server to use the correct encryption key when sending a message to a particular client.

Conceptually:

```
Client 1 ——— Key K1 ——— Server  
Client 2 ——— Key K2 ——— Server  
  
K1 ≠ K2
```

This prevents the server from accidentally using the same symmetric key for different client connections.

4. Deriving the AES Key

The raw DH shared secret is not directly used as the AES key.

After computing the shared secret, it is converted into a fixed-length byte representation and hashed using SHA-256:

```
EVP_DigestInit_ex(mdctx, EVP_sha256(), nullptr);  
EVP_DigestUpdate(mdctx, secret_bytes.data(), secret_bytes.size());  
EVP_DigestFinal_ex(mdctx, aes_key.data(), &hash_len);
```

The SHA-256 output is 32 bytes, giving a 256-bit AES key.

Therefore:

```
DH Shared Secret  
  |  
  ▼  
SHA-256  
  |  
  ▼  
256-bit AES Key
```

Hashing the shared secret provides a fixed-length, uniformly represented key suitable for use with AES. It also avoids directly treating the mathematical DH value as a symmetric encryption key.

5. AES-GCM Encryption

After the DH exchange, subsequent communication is encrypted using **AES-256-GCM**.

For every message, a fresh 12-byte random nonce is generated:

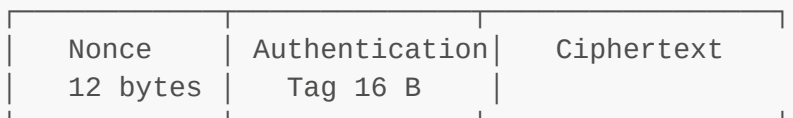
```
unsigned char nonce[12];

RAND_bytes(nonce, sizeof(nonce));
```

The plaintext is then encrypted using AES-256-GCM:

```
EVP_EncryptInit_ex(
    ctx,
    EVP_aes_256_gcm(),
    nullptr,
    aes_key.data(),
    nonce
);
```

The resulting network payload contains:



The authentication tag is generated by GCM and is later used to verify that the ciphertext has not been modified.

This provides both confidentiality and integrity for the encrypted messages.

6. Encrypted Login and Chat Communication

Unlike Phase 1, the username exchange is also encrypted after the DH handshake.

The server encrypts its username prompt:

```
std::vector<unsigned char> enc_prompt = dh.encrypt(prompt);
```

The client decrypts the prompt using the derived AES key.

Similarly, the username entered by the client is encrypted before being sent:

```
std::vector<unsigned char> enc_username = dh.encrypt(username);
```

Normal chat commands such as:

```
@client2 Hello  
/who  
/quit
```

are also encrypted before transmission.

Therefore, after the DH exchange, the application does not send the actual chat content as plaintext over the network.

7. DH Fingerprint Verification

To verify that both sides derived the same shared key, a hash-based fingerprint is generated.

The fingerprint is calculated from the derived AES key using SHA-256, and only the first eight bytes of the resulting hash are displayed.

For example:

```
DH Fingerprint: cf5022078beb8bbe
```

The raw DH shared secret and AES key are never printed.

During testing, the fingerprint displayed by the client matched the fingerprint displayed by the server for the corresponding connection.

This demonstrates that both endpoints independently derived the same symmetric key.

Figure 1: Matching DH fingerprints on client and server

[INSERT DH FINGERPRINT SCREENSHOT HERE]

8. Wireshark Verification

The same traffic inspection performed in Phase 1 was repeated after introducing encryption.

The TCP traffic was captured using Wireshark on port 1111 and inspected using:

```
Follow → TCP Stream
```

Unlike Phase 1, the actual chat message is no longer visible in plaintext.

Instead, the TCP stream contains binary-looking encrypted data corresponding to the nonce, authentication tag, and ciphertext.

Figure 2: Wireshark capture showing encrypted Phase 2 traffic

[INSERT WIRESHARK SCREENSHOT HERE]

This demonstrates that a passive network observer can no longer directly read the application message from the captured TCP stream.

9. AES-GCM Tamper Detection

AES-GCM also provides authentication of the encrypted data.

To test this property, a single byte of the ciphertext was modified before attempting decryption.

The decryption function verifies the GCM authentication tag using:

```
EVP_CIPHER_CTX_ctrl(  
    ctx,  
    EVP_CTRL_GCM_SET_TAG,  
    16,  
    tag  
);
```

If the ciphertext has been modified, the authentication check fails:

```
if (ret <= 0) {  
    return "<DECRYPTION_FAILED_TAMPERING_DETECTED>";  
}
```

Therefore, modified ciphertext is rejected instead of being silently decrypted into corrupted plaintext.

Figure 3: AES-GCM tampering test

[INSERT TAMPERING TEST SCREENSHOT HERE]

The observed result was:

```
<DECRYPTION_FAILED_TAMPERING_DETECTED>
```

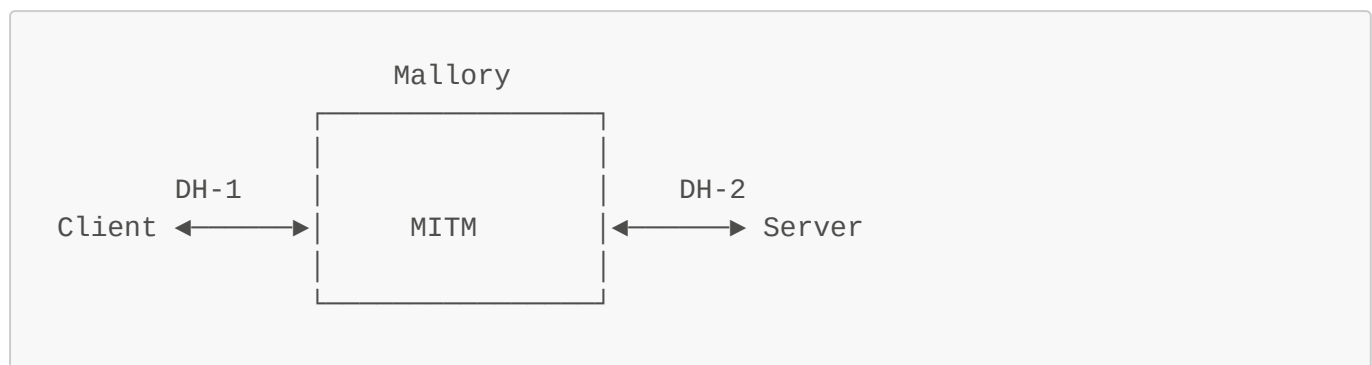
This confirms that the integrity protection provided by AES-GCM is functioning correctly.

10. Man-in-the-Middle Attack

Although Diffie-Hellman protects the communication from passive eavesdropping, the basic DH exchange does not authenticate the party on the other side.

To demonstrate this weakness, a separate `mitm.cpp` program was implemented.

The proxy establishes two independent DH exchanges:



Mallory therefore has:

Client ↔ Mallory : Key K1

Mallory ↔ Server : Key K2

The client believes that it has established a secure key with the server, while the server believes that it has established a secure key with the client.

In reality, Mallory possesses both keys.

11. MITM Message Interception

The proxy decrypts messages received from the client using the client-facing key:

```
std::string plaintext = dh_client->decrypt(payload);
```

The plaintext is then logged:

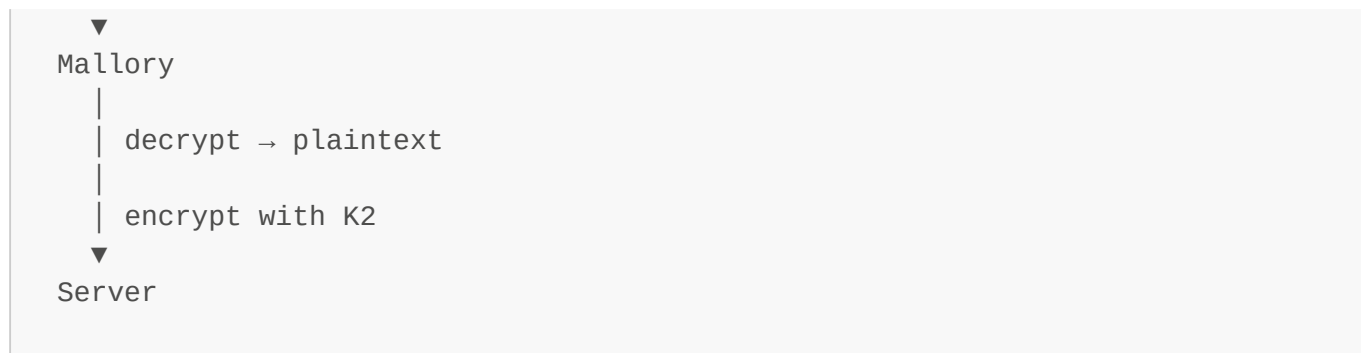
```
std::cout << "\n[INTERCEPT C->S]: "  
          << plaintext << std::endl;
```

Mallory subsequently encrypts the plaintext using the server-facing key and forwards it to the real server.

The same process is performed in the reverse direction.

Thus:

```
Client  
|  
| encrypted with K1
```



This allows Mallory to read the supposedly protected communication while still allowing the client and server to communicate normally.

Figure 4: MITM proxy successfully intercepting plaintext

[INSERT MITM LOG SCREENSHOT HERE]

Example output:

```
[MITM] Handshakes complete. Logging traffic...

[INTERCEPT C->S]: @client2 Hello from client1

[INTERCEPT S->C]: [client2] says: Hello from client1
```

This demonstrates that unauthenticated Diffie–Hellman alone does not provide protection against an active MITM attacker.

12. Phase 2 Result

Phase 2 successfully adds confidentiality and authenticated encryption to the client-server communication.

Each client independently performs a Diffie–Hellman exchange with the server, derives a shared secret, and converts that secret into a 256-bit AES key using SHA-256. Subsequent communication is protected using AES-256-GCM.

The matching DH fingerprints confirm that both endpoints derive the same key, while the Wireshark capture demonstrates that application messages are no longer visible as plaintext.

The AES-GCM tampering test further demonstrates that modification of encrypted data is detected.

However, the MITM experiment demonstrates an important limitation: **Diffie-Hellman by itself does not authenticate the identity of the peer**. Mallory can establish separate keys with the client and server and transparently relay decrypted messages between them.

This limitation motivates the authentication mechanism introduced in Phase 3.

Phase 3 – Server Authentication via PKI

Objective

The objective of Phase 3 is to authenticate the chat server using Public Key Infrastructure (PKI). The client validates the server certificate using a trusted Certificate Authority (CA) before proceeding with the Diffie-Hellman exchange.

The server also proves that it possesses the private key corresponding to the public key contained in the certificate.

1. Certificate Authority and Server Certificate Setup

A private Certificate Authority was created using OpenSSL. The CA consists of a private key and a self-signed root certificate.

The server key pair was then generated, followed by the creation of a Certificate Signing Request (CSR). The CSR was signed using the CA private key to generate the server certificate.

The main files generated are:

```
certs/  
├─ ca.key  
├─ ca.crt  
├─ server.key  
└─ server.crt
```

The commands used were:

```
# Generate CA private key  
openssl genrsa -out certs/ca.key 4096  
  
# Generate self-signed CA certificate  
openssl req -x509 -new -nodes \  
  -key certs/ca.key \  
  -sha256 -days 3650 \  
  -out certs/ca.crt \  
  -subj "/C=IN/ST=Maharashtra/L=Mumbai/O=SecureChat/OU=CA/CN=SecureChat  
Root CA"  
  
# Generate server private key  
openssl genrsa -out certs/server.key 2048  
  
# Generate server CSR  
openssl req -new \  
  -key certs/server.key \  
  -out certs/server.csr \  
  -subj  
"/C=IN/ST=Maharashtra/L=Mumbai/O=SecureChat/OU=Server/CN=SecureChat Server"  
  
# Sign server CSR using the CA
```

```
openssl x509 -req \  
  -in certs/server.csr \  
  -CA certs/ca.crt \  
  -CAkey certs/ca.key \  
  -CAcreateserial \  
  -out certs/server.crt \  
  -days 365 \  
  -sha256
```

Screenshot – OpenSSL Certificate Generation

[INSERT SCREENSHOT HERE]

2. Updated Client and Server

The Phase 2 client and server were modified to implement certificate authentication.

Certificate Exchange

Before performing Diffie-Hellman, the server sends its certificate to the client.

The client receives and parses the certificate using OpenSSL and validates it against its trusted copy of:

```
certs/ca.crt
```

The client verifies that:

- The certificate is signed by the trusted CA.
- The certificate is within its validity period.
- The certificate is trusted by the client's CA store.

If validation fails, the client immediately closes the connection and does not proceed with the remaining handshake.

Proof-of-Possession

After successful certificate validation, the client generates a random 32-byte challenge.

```
Client → Server : Random Challenge
```

The server signs this challenge using:

```
certs/server.key
```

The signature is sent back to the client.

The client extracts the public key from the validated server certificate and verifies the signature.

```
Client → Server : Challenge
Client ← Server : Signature(Challenge)
Client       : Verify using certificate public key
```

Only after successful proof-of-possession does the client proceed with the Diffie-Hellman exchange.

Screenshot – Legitimate Phase 3 Connection

[INSERT SCREENSHOT HERE]

The screenshot should show:

```
Received server certificate
Server certificate validation SUCCESS
Challenge sent to server
Received server signature
Server proof-of-possession SUCCESS
DH Fingerprint: ...
```

3. Updated MITM Attack

The Phase 2 MITM proxy was modified to attack the Phase 3 protocol.

Mallory generates a different certificate that is not signed by the trusted CA and sends it to the victim client.

The client receives Mallory's certificate and attempts to validate it using its trusted CA certificate.

Since Mallory's certificate is not signed by the trusted CA, certificate validation fails.

The client immediately terminates the connection.

The MITM therefore cannot continue to the Diffie-Hellman exchange.

Expected Result

```
[MITM] Sent Mallory certificate to victim.
[MITM] Waiting for victim to validate certificate...

SERVER CERTIFICATE VALIDATION FAILED
Reason: unable to get local issuer certificate
```

The MITM observes that the victim closes the connection.

Screenshot – MITM Attack Failure

[INSERT SCREENSHOT HERE]

4. Proof-of-Possession Bypass Attempt

A second attack was performed where Mallory possesses a copy of the legitimate server certificate but does not possess the corresponding private key.

Mallory forwards the legitimate certificate to the client but signs the client's random challenge using a different private key.

The client verifies the signature using the public key contained in the legitimate server certificate.

Since Mallory's private key does not correspond to that public key, signature verification fails.

The client therefore rejects the connection.

Expected Result

```
Received server signature
SERVER PROOF-OF-POSSESSION FAILED
```

The MITM observes that the victim closes the connection.

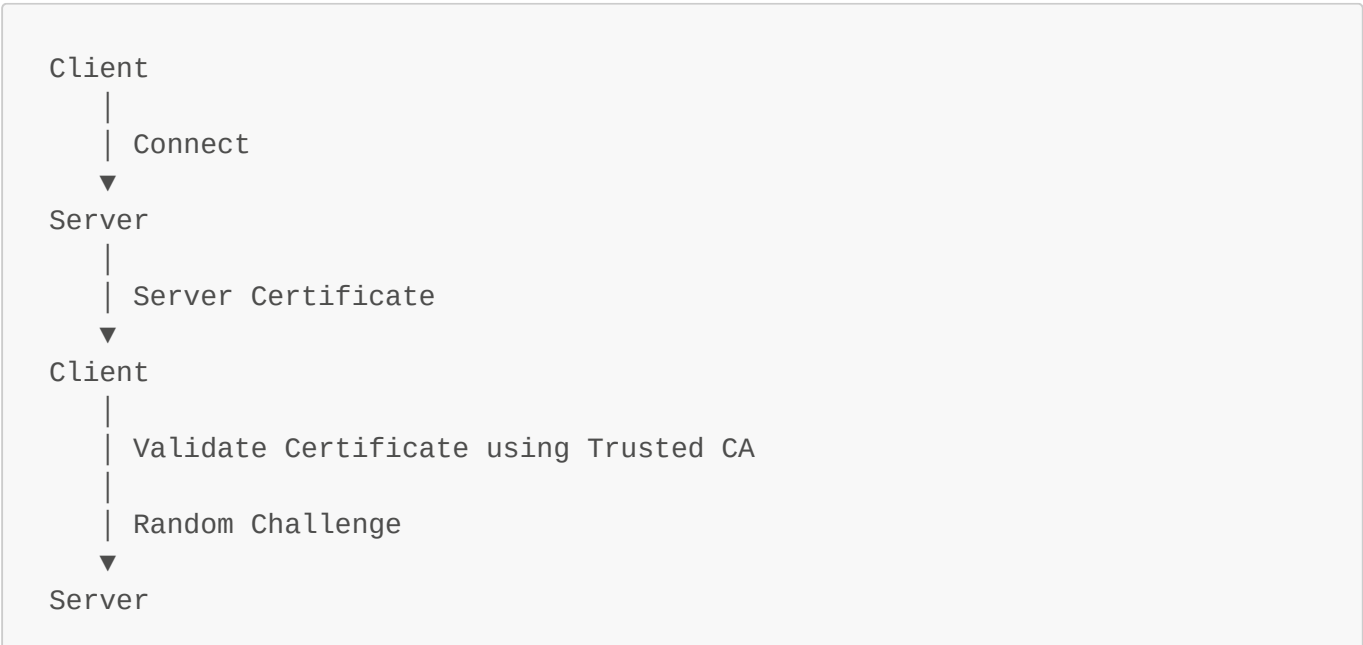
Screenshot – Proof-of-Possession Failure

[INSERT SCREENSHOT HERE]

6. Final Result

Phase 3 successfully implements server authentication using PKI.

The final secure handshake is:





The Phase 2 MITM attack is successfully prevented because an attacker cannot authenticate as the server without a certificate signed by the trusted CA and the corresponding server private key.